

ML for Data Science

Model Selection

Dr. Vinny Davies

(original slides by Dr Simon Rogers)

vinny.davies@glasgow.ac.uk

Model selection

- Our definition
 - Any optimisation done **outside the training process**.

Example: SVMs

- Finding alpha is **training**
 - Choosing the kernel is **model selection**
-
- Choosing SVMs in the first place is also model selection!
 - Why do you choose a particular method? - We will come to this in a minute

Other Examples

- Choosing a particular family of models (SVM versus KNN etc)
- KNN: choosing K
- Kmeans: choosing K
- Gaussian Process: choosing covariance function (and its hyper-parameters)
- Linear regression: choosing polynomial order
- Regularised linear regression: choosing regularisation parameter
- Random forest: choosing the number of trees

Can you think of any other examples?

Choosing a family of models

Choosing a method

- Choosing a method is not normally a straightforward procedure
- How would we go about choosing a method to use?

Problem and Data

- What are the aims of the project? E.g. Explainability or Prediction
- What data do we have available? E.g. Regression or Clustering
- What type of data do we have?
- How much data do we have?
- Do we expect to see non-linear relationships?
- Are there any regulatory constraints?
- What end results do we want?
- What performance measure(s) are we trying to satisfy?
- What prior knowledge do we have?

Practical Constraints

- Computational time
- Computational power
- Ease of implementation
- Explainability
- Allocated modelling time (e.g. how many hours can you spend on the project?)

Compare against a baseline

- There is a tendency to just fit Neural Networks etc, but how do we know whether the performance is good?
 - Comparing against a simple baseline where possible, e.g. linear regression, helps us see whether these more complex methods are worthwhile
 - Complex methods usually come at some form of cost in terms of interpretability
-
- What happens if there is minimal performance improvement?
 - Should we consider using the simple method instead?
 - Think about our other constraints beyond best performance

Reduce your options down to a few

- It's impossible to fit every model to a dataset
- Reduce your possible models down to a few
 - What I am calling a 'family of models'
- These models will hopefully meet most or all of your criteria
- For each model we will still need to do **training** and **model selection**
 - This is just the first step of model selection

Choosing from a group of models

Choosing from a group of models

- We can be either
 - Choosing between variations of one model, e.g. choosing a kernel
 - Choosing between different models, e.g. Neural Net or SVM
 - Or choosing both of the above
- On what criteria do we choose?
 - Predictive performance? MSE or MAE?
 - Classification performance?
 - Interpretability?

Predictions

- Assume we are building an ML system for a **predictive** task
- The best model is the model that gives the **best predictions**
- Define best:
 - Highest accuracy (is this always good?)
 - Lowest number of false negatives (diagnostics)
 - ...best will be **application dependent**
- Model selection scheme:
 - Train each of the models in question
 - Test their predictions against ground truth
 - Pick the best one

Predictions on training data

- Can we use predictions on the training data?
- NO!

Predictions on training data

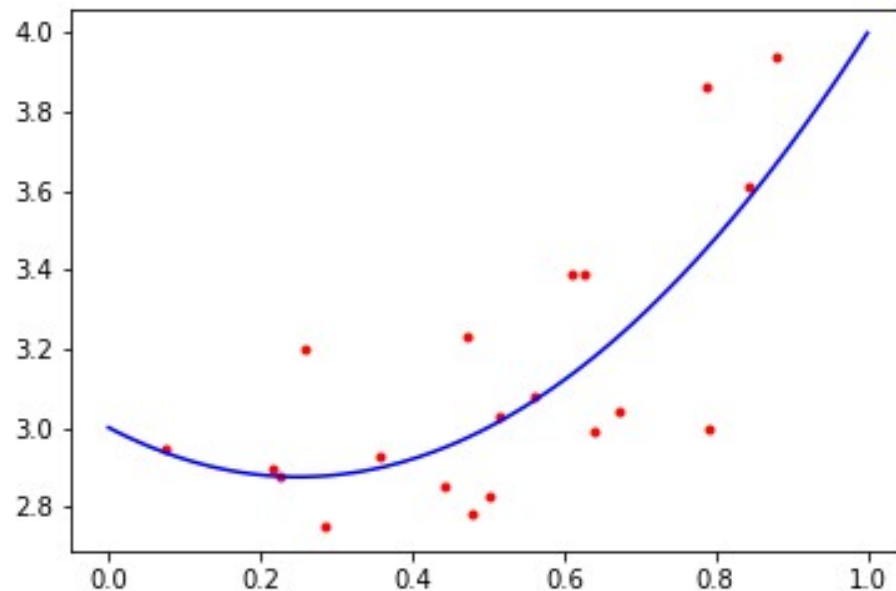
- Can we use predictions on the training data?
- NO!
- Often our model selection choice will be between models of varying **complexity** in the same family (polynomial order, SVM kernel hyper-parameter)
- As a model becomes more complex, it can **memorise** more of the training data
- Being able to memorise more does not necessarily mean we can predict better (generalisation)
-**memorising** answers to past exam questions doesn't help you much

Out of sample data

- Techniques you've already seen (holding out a validation set, Cross-Validation) should be used instead of training predictions
- We can see this in a similar linear regression example
- Keep an eye out for what happens if we look at the training data and the held out data

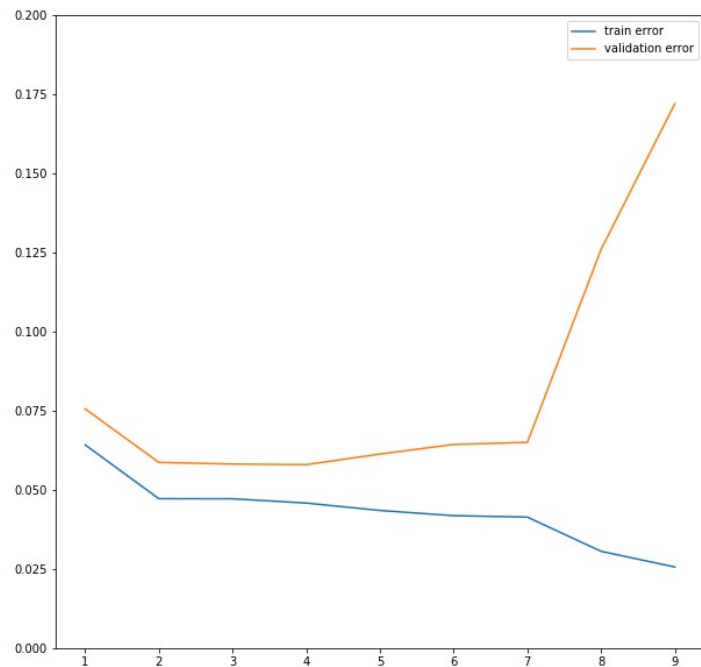
Data and true function

- Quadratic
- $w_0 = 3, w_1 = -1, w_2 = 2$
- Noise variance = 0.05

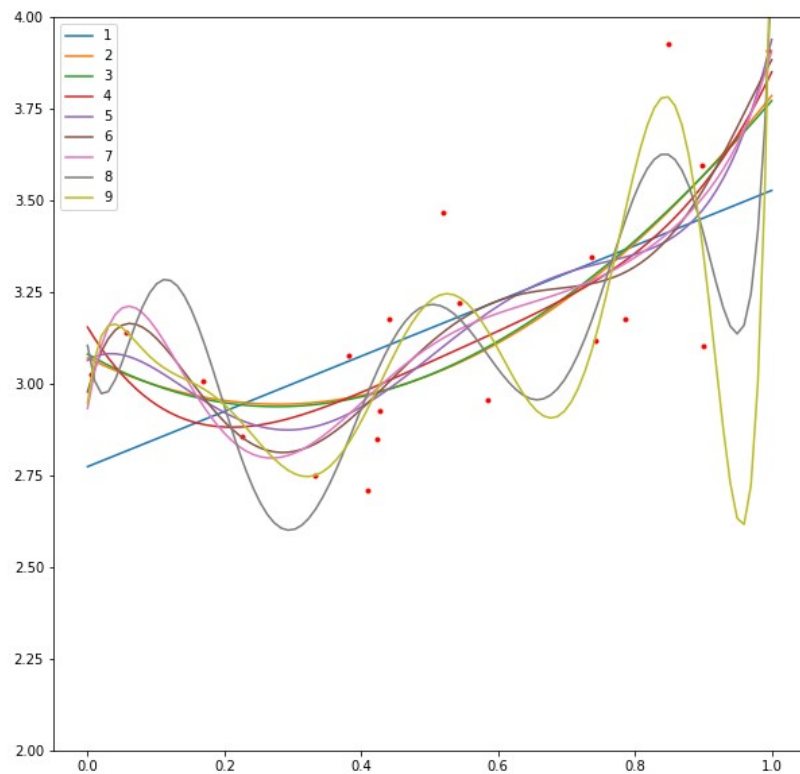


Training error v validation error (validation N = 100)

- Mean squared error as polynomial order increases
- Training error will **always** decrease as complexity increases
 - Memorising v generalising
 - Over-fitting
- Validation error is minimised between ~2 and 4
- Which one should we pick?



What do the functions look like?



Choosing validation set

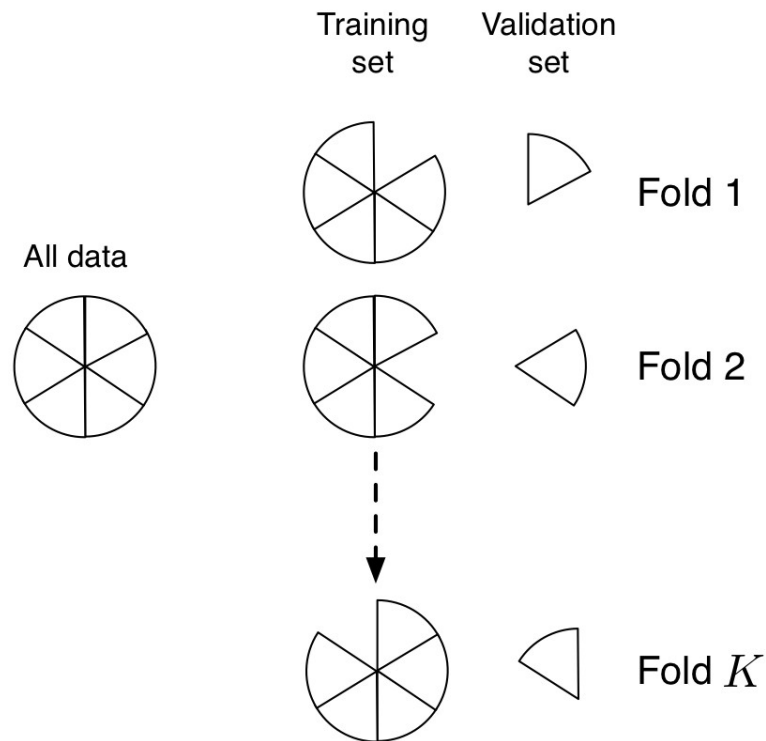
- Most of the time, **random selection** is best as it avoids bias
- To do this we need to
 - Choose how much to keep as training data
 - And how much to use as validation data
- Once we have choose this we can randomly assign data to either be training or validation

Choosing validation set

- When might bias be important?
- Time series: normally predicting the future is more important than predicting the past
- When data we're predicting might come from a slightly different distribution than training
 - E.g. clinical data collected from hospitals A, B, C, and D that will be train a model to be used at a new hospital, E.
 - Validate by holding out a hospital (batch)

What if we don't have a nice validation set?

- Data is almost always a limiting factor
- How much to train? How much to validate?
 - More train = better model, harder to validate
 - More validate = worse model, but easy to measure how bad it is!
- Cross-validation:
 - Repeatedly hold out a chunk of data
 - Limit -- with N observations, train N different models, each with one point held-out for testing



In-sample data

- Can't use the quantity we're optimising
 - Likelihood, loss
- Increasing complexity will generally increase this value: **overfitting**
- But: there exist quantities that we **can** use
 - They combine the likelihood with a term than penalises complexity
- Akaike Information Criteria (AIC)
- Bayesian Information Criteria (BIC)

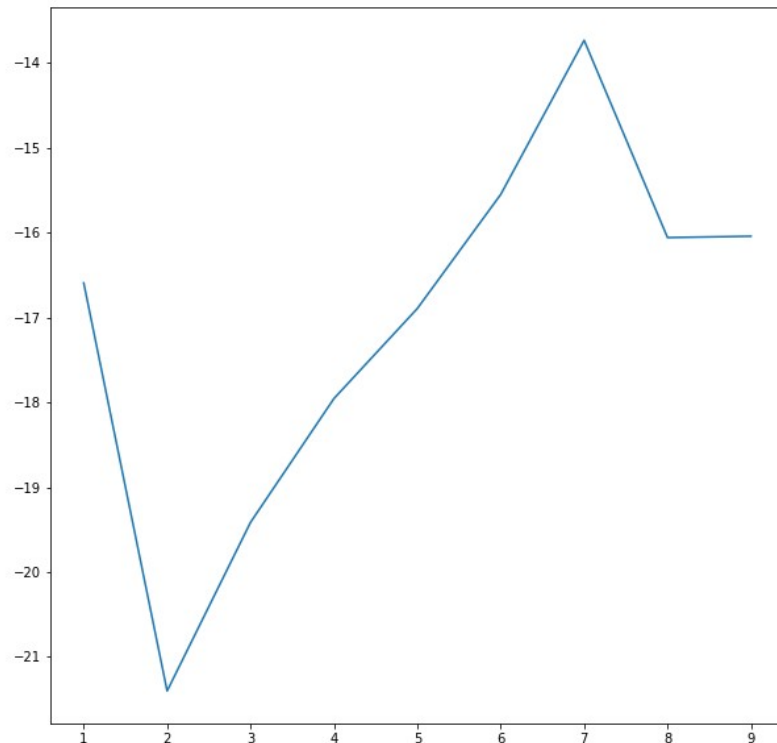
AIC

$$AIC = 2k - \ln \hat{L}$$

- k = number of parameters
- L = value of likelihood at maxima
- Chose the model with lowest AIC
- We can plot it for our polynomial example...
- Likelihood is product over training points of normal pdf function (because we assumed normal noise)

AIC

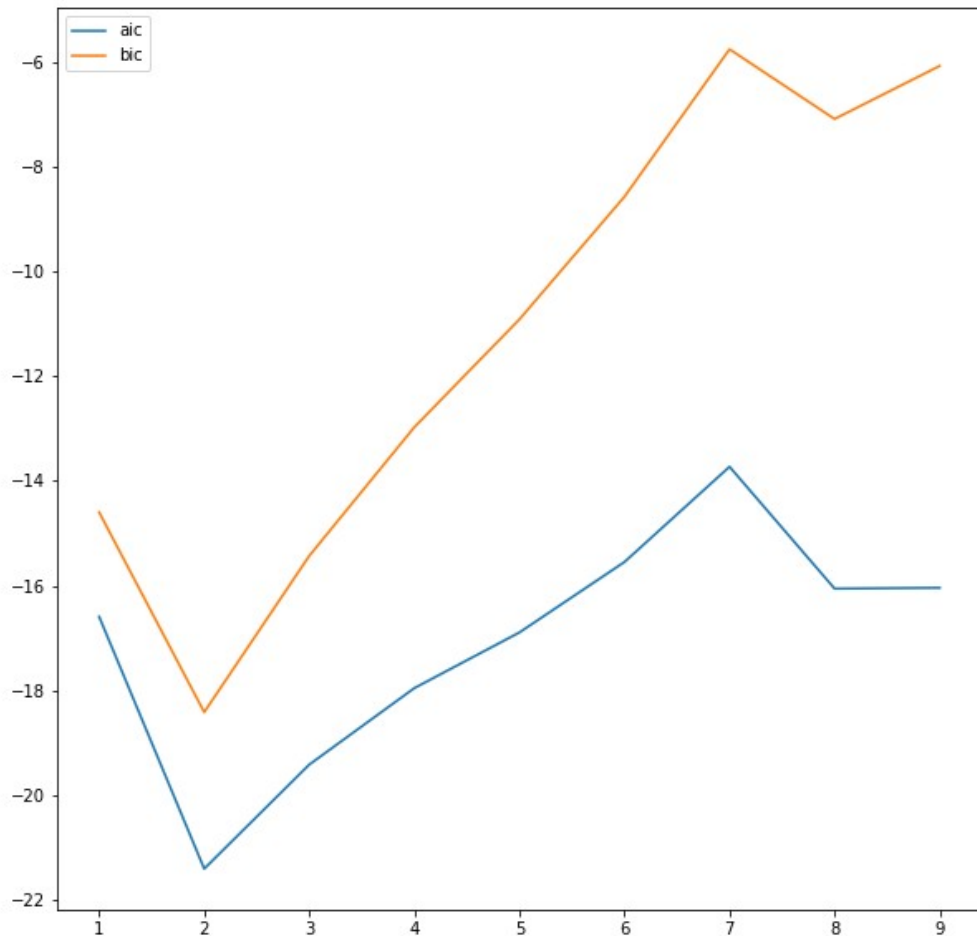
- AIC correctly suggests a quadratic (order 2) model
- But: it will only work when maximising likelihood (and not, say, minimising some loss)
- In this example I assumed we knew the noise variance.
- If we didn't know that, it probably wouldn't be so clear...



BIC

$$BIC = k \ln(n) - 2 \ln \hat{L}$$

- Very similar but penalty has $\ln(n)$ instead of 2
- n = number of training points
- Low BIC is good
- Increases more rapidly as complexity increases
 - Needs more data to be confident that extra complexity is warranted



AIC v BIC

- When should you use {A,B}IC?
- **In theory** BIC is better if the **true** model is in the set being compared
- What is the **true** model?
 - In our example there is a true model. Quadratic, known noise variance
 - In practice, the **true** model doesn't really exist
- If you can compute one, you can compute the other...
 - Try them both!

Let's get philosophical

- Is BIC **really** Bayesian?
- BIC is computed for a particular set of parameter values
 - Those that maximise the likelihood
- Bayesians don't like parameter values
 - They average over them
- For model comparisons, the Bayesian holy grail is the **marginal likelihood**
 - But..in practice, it's often impossible to calculate...

Bayesian inference

$$p(w|X, y, \theta) = \frac{p(y|w, X, \theta)p(w)}{p(y|X)}$$

- Assume linear regression (although it's general)
- w = regression parameters
- X = train data, y = train labels, θ = anything else (e.g. noise variance)
- Equation above links the posterior (LHS) with the likelihood and prior (RHS), and marginal likelihood (RHS, denominator)

$$p(w|X, y, \theta) = \frac{p(y|w, X, \theta)p(w)}{p(y|X)}$$

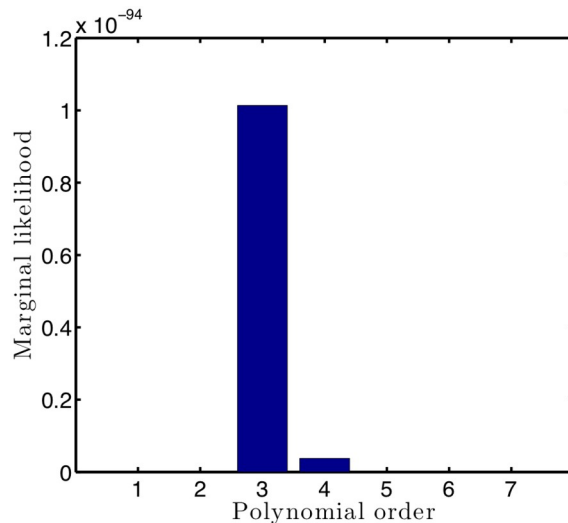
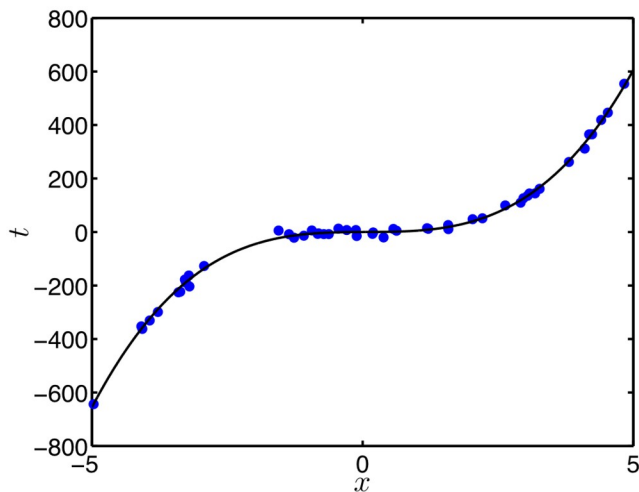
Marginal likelihood

- Marginal likelihood (RHS; denominator) is computed by averaging out the parameters from the numerator of the RHS.
- This is almost always **analytically** intractable.
 - An exception is when the likelihood and prior form a conjugate pair, e.g. Gaussian and Gaussian
- Numerically approximating it is **very challenging**...
- ...it's a high-dimensional integral

$$p(y|X) = \int p(y|w, X, \theta)p(w)dw$$

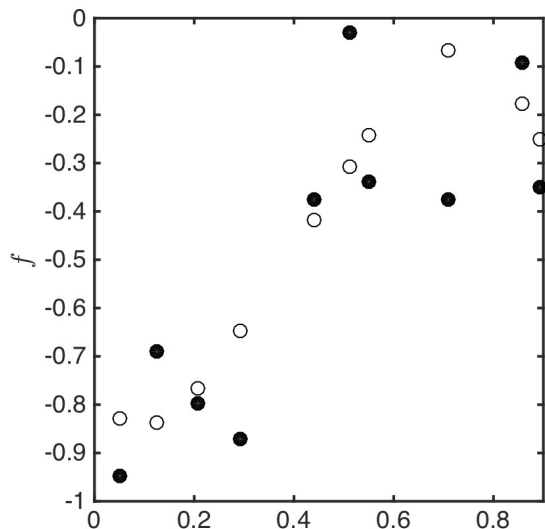
Marginal likelihood when conjugate

- Assume prior and likelihood are both Gaussian.
- Marginal likelihood is another Gaussian
- Evaluate for different orders
- Example of cubic (third order) taken from <http://bit.ly/firstcourseml>

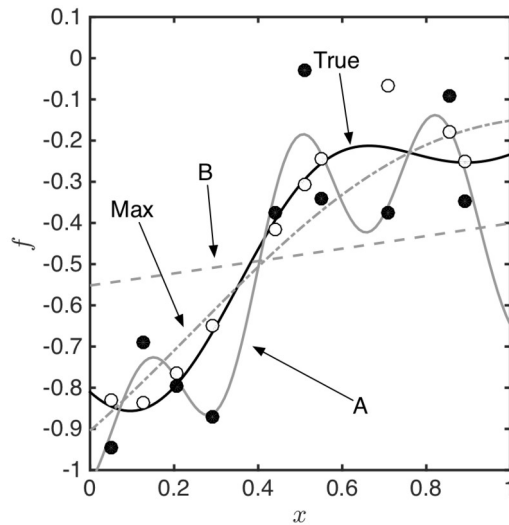
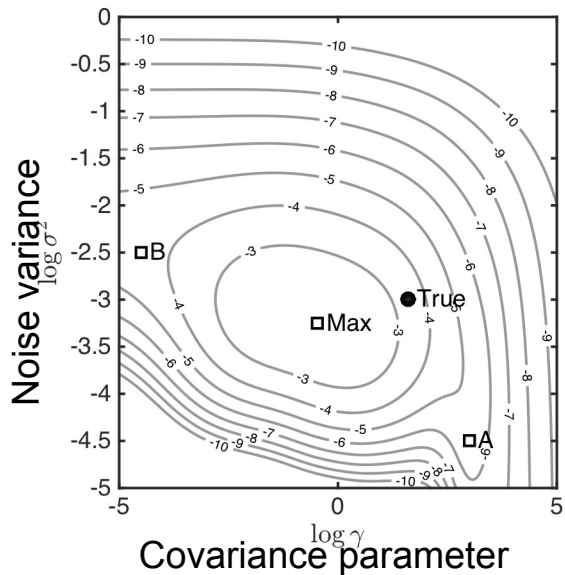


Aside - Gaussian Processes

- You saw GPs earlier in the semester
- Under certain conditions (regression, gaussian noise), GP marginal likelihood is analytically tractable and can be used to optimise parameters
- Example of optimising noise variance and covariance parameter



White = true function
black = noisy observations



Finally: ensemble methods

- Why do we want to choose a single model anyway?
- Ensemble methods fit multiple methods, then combine them to get to get predictions

Summary

Model selection:

- Any optimisation that is not **training**

Strategies (in rough order of popularity):

- Out of sample performance: e.g. validation, CV, LOO
- In sample performance: e.g. AIC, BIC
- Being Bayesian: compute marginal likelihood

Generally an ill-posed problem:

- What does *best* mean?